

Database schema

Gaia Costing

Product costing and daily production cost - PostgreSQL

Every table behind the costing app, grouped by what it is for. The model half holds the rate card - what a unit should cost at a given set of rates. The daily half holds what the plant actually spent, day by day, and what that made each product cost.

	Tables	Columns	What it covers
The cost model	9	128	The rate card: what a unit should cost.
Daily production	6	52	What a day actually cost.
Calculation and audit	3	25	How the numbers were reached, and who changed what.
Reference and access	2	13	Units, and the people who can sign in.
Views	2	-	Flattened reads over the tables above.

20 tables, 218 columns. Row counts are as at generation time.

The cost model

The rate card: what a unit should cost. Nothing here is fixed by the schema - the kinds of things you cost, the columns each carries, and the formulas behind them are all rows you can change while the app runs.

scenario

One complete, versioned cost model - a workbook. Clone it to run a what-if without touching the live numbers. 1 row

Column	Type	Req	Default	Key	Notes
id	uuid	yes	gen_random_uuid()	PK	
code	text	yes		unique	
name	text	yes			
description	text				
effective_from	date	yes	CURRENT_DATE		
status	text	yes	'draft'		
currency	text	yes	'INR'		
cloned_from_id	uuid			FK to scenario	
created_by	uuid			FK to app_user	
created_at	timestampz	yes	now()		
updated_at	timestampz	yes	now()		
UNIQUE (code) · CHECK ((status = ANY (ARRAY['draft'::text, 'active'::text, 'archived'::text])))					

parameter

Every rate, percentage and factor the model leans on: base milk rate, GST, commission, markups. A parameter may itself be a formula over other parameters. 40 rows

Column	Type	Req	Default	Key	Notes
id	uuid	yes	gen_random_uuid()	PK	
scenario_id	uuid	yes		FK to scenario, unique	
key	text	yes		unique	
label	text	yes			
group_name	text	yes	'General'		
value_num	numeric(20,8)				
formula	text				when set, overrides value_num
uom_id	uuid			FK to uom	
min_value	numeric(20,8)				
max_value	numeric(20,8)				
decimals	integer	yes	4		
suffix	text				
description	text				
is_locked	boolean	yes	false		
sort_order	integer	yes	0		
computed_num	numeric(20,8)				cached result of the last calculation
computed_at	timestampz				
error	text				
created_at	timestampz	yes	now()		
updated_at	timestampz	yes	now()		
UNIQUE (scenario_id, key) · CHECK ((key ~ '^[a-z][a-z0-9_]*\$'::text))					

object_class

The kinds of costed things - milk batch, product, packaging, ingredient, recipe. Add your own without a migration. 5 rows

Column	Type	Req	Default	Key	Notes
id	uuid	yes	gen_random_uuid()	PK	
scenario_id	uuid	yes		FK to scenario, unique	
code	text	yes		unique	
name	text	yes			
plural_name	text				
description	text				
allows_bom	boolean	yes	true		
cost_field	text				column holding this class's unit cost
is_locked	boolean	yes	false		
color	text				
sort_order	integer	yes	0		
created_at	timestamptz	yes	now()		
updated_at	timestamptz	yes	now()		
UNIQUE (scenario_id, code) · CHECK ((code ~ '^[a-z][a-z0-9_]*\$'::text)					

field_group

Groups columns for display, and gives roll-ups something to sum into. 11 rows

Column	Type	Req	Default	Key	Notes
id	uuid	yes	gen_random_uuid()	PK	
scenario_id	uuid	yes		FK to scenario	
class_id	uuid	yes		FK to object_class, unique	
code	text	yes		unique	
label	text	yes			
sort_order	integer	yes	0		
UNIQUE (class_id, code)					

field_def

The columns of a class. Each carries a default formula or a default value; any single row may override either. 38 rows

Column	Type	Req	Default	Key	Notes
id	uuid	yes	gen_random_uuid()	PK	
scenario_id	uuid	yes		FK to scenario	
class_id	uuid	yes		FK to object_class, unique	
key	text	yes		unique	
label	text	yes			
data_type	text	yes	'number'		
group_id	uuid			FK to field_group	
uom_id	uuid			FK to uom	
default_value	numeric(20,8)				
default_text	text				
default_formula	text				used by every row that does not override it

Column	Type	Req	Default	Key	Notes
rollup_group	text				sums every column carrying this tag
is_total	boolean	yes	false		renders as a total column
is_locked	boolean	yes	false		
is_active	boolean	yes	true		
decimals	integer	yes	2		
prefix	text				
suffix	text				
width	integer				
description	text				
sort_order	integer	yes	0		
created_at	timestamptz	yes	now()		
updated_at	timestamptz	yes	now()		

UNIQUE (class_id, key) · CHECK ((data_type = ANY (ARRAY['number'::text, 'text'::text, 'boolean'::text, 'date'::text, 'reference'::text])) · CHECK ((key ~ '^[a-z][a-z0-9_]*\$'::text)

field_rollup_tag

Tags a column as feeding a total. One column can feed several totals, and a negative sign subtracts it. 8 rows

Column	Type	Req	Default	Key	Notes
field_def_id	uuid	yes		PK, FK to field_def	
tag	text	yes		PK	
sign	numeric(4,2)	yes	1		

cost_object

The rows: a SKU, a batch type, a carton, a recipe. 96 rows

Column	Type	Req	Default	Key	Notes
id	uuid	yes	gen_random_uuid()	PK	
scenario_id	uuid	yes		FK to scenario, unique	
class_id	uuid	yes		FK to object_class	
code	text	yes		unique	
name	text	yes			
parent_id	uuid			FK to cost_object	for variants of a parent item
uom_id	uuid			FK to uom	
is_active	boolean	yes	true		
notes	text				
tags	text[]	yes	'{}'[]		
sort_order	integer	yes	0		
created_at	timestamptz	yes	now()		
updated_at	timestamptz	yes	now()		

UNIQUE (scenario_id, code) · CHECK ((code ~ '^[a-z][a-z0-9_]*\$'::text)

field_value

The cells. A row's own formula wins, then its own literal, then the column's formula, then the column's default. 1064 rows

Column	Type	Req	Default	Key	Notes
id	uuid	yes	gen_random_uuid()	PK	
object_id	uuid	yes		FK to cost_object, unique	
field_def_id	uuid	yes		FK to field_def, unique	
formula	text				row-level override of the column formula
value_num	numeric(20,8)				row-level literal; beats the column formula
value_text	text				
value_bool	boolean				
value_ref	uuid			FK to cost_object	
computed_num	numeric(20,8)				cached result, so the sheet renders without recalculating
computed_text	text				
computed_at	timestampz				
error	text				
updated_at	timestampz	yes	now()		
UNIQUE (object_id, field_def_id)					

bom_line

Recipe and packaging build-ups: component, quantity, rate, divisor. Amount is qty x rate / divisor unless a formula says otherwise. 95 rows

Column	Type	Req	Default	Key	Notes
id	uuid	yes	gen_random_uuid()	PK	
scenario_id	uuid	yes		FK to scenario	
parent_object_id	uuid	yes		FK to cost_object	
component_object_id	uuid			FK to cost_object	
line_type	text	yes	'input'		
label	text				
qty	numeric(20,8)		1		
qty_formula	text				
uom_id	uuid			FK to uom	
rate	numeric(20,8)				
rate_formula	text				
divisor	numeric(20,8)	yes	1		units per pack - a carton of 12 divides by 12
divisor_formula	text				
loss_pct	numeric(10,6)	yes	0		wastage added on top of qty x rate
amount_formula	text				
include_in_total	boolean	yes	true		
computed_qty	numeric(20,8)				resolved quantity, reused by daily costing
computed_rate	numeric(20,8)				
computed_divisor	numeric(20,8)				
computed_amount	numeric(20,8)				
computed_at	timestampz				

Column	Type	Req	Default	Key	Notes
error	text				
notes	text				
sort_order	integer	yes	0		
created_at	timestampz	yes	now()		
updated_at	timestampz	yes	now()		
CHECK ((line_type = ANY (ARRAY['input'::text, 'packaging'::text, 'labour'::text, 'overhead'::text, 'transport'::text, 'additive'::text, 'credit'::text, 'oth					
· CHECK (((component_object_id IS NULL) OR (component_object_id <> parent_object_id))					

Daily production

What a day actually cost. The day supplies the milk processed and the shared costs; divide one by the other and every product carries that rate in proportion to the milk inside it.

production_day

A day of production. milk_processed_l is the divisor for every shared cost that day. 1 row

Column	Type	Req	Default	Key	Notes
id	uuid	yes	gen_random_uuid()	PK	
scenario_id	uuid	yes		FK to scenario, unique	
day	date	yes		unique	
milk_processed_l	numeric(20,4)				divisor for every shared cost that day
status	text	yes	'open'		
notes	text				
created_at	timestamptz	yes	now()		
updated_at	timestamptz	yes	now()		

UNIQUE (scenario_id, day) · CHECK ((status = ANY (ARRAY['open'::text, 'closed'::text]))

overhead_head

The shared cost heads the plant spends on daily - coal, power, labour, transport. Add or remove freely. 7 rows

Column	Type	Req	Default	Key	Notes
id	uuid	yes	gen_random_uuid()	PK	
scenario_id	uuid	yes		FK to scenario, unique	
code	text	yes		unique	
label	text	yes			
unit	text				
is_active	boolean	yes	true		
notes	text				
sort_order	integer	yes	0		
created_at	timestamptz	yes	now()		
updated_at	timestamptz	yes	now()		

UNIQUE (scenario_id, code) · CHECK ((code ~ '^[a-z][a-z0-9_]*\$'::text)

daily_overhead

What each head cost that day. Enter an amount, or a quantity and a rate. 5 rows

Column	Type	Req	Default	Key	Notes
id	uuid	yes	gen_random_uuid()	PK	
day_id	uuid	yes		FK to production_day, unique	
head_id	uuid	yes		FK to overhead_head, unique	
qty	numeric(20,4)				
rate	numeric(20,4)				
amount	numeric(20,4)				entered directly, or qty x rate
notes	text				
UNIQUE (day_id, head_id)					

daily_production

What was produced that day, per product. 5 rows

Column	Type	Req	Default	Key	Notes
id	uuid	yes	gen_random_uuid()	PK	
day_id	uuid	yes		FK to production_day, unique	
product_id	uuid	yes		FK to cost_object, unique	
qty_produced	numeric(20,4)	yes	0		
notes	text				
UNIQUE (day_id, product_id)					

daily_material_use

Raw material actually drawn for a product that day, when it differed from the recipe. 0 rows

Column	Type	Req	Default	Key	Notes
id	uuid	yes	gen_random_uuid()	PK	
day_id	uuid	yes		FK to production_day, unique	
product_id	uuid	yes		FK to cost_object, unique	
material_id	uuid			FK to cost_object, unique	
label	text				
qty	numeric(20,6)				
rate	numeric(20,6)				
amount	numeric(20,4)				
UNIQUE (day_id, product_id, material_id)					

daily_product_cost

The frozen answer: what each product cost that day, split into material, conversion, packing and other. Written when the day is saved, so later rate changes never rewrite history. 5 rows

Column	Type	Req	Default	Key	Notes
id	uuid	yes	gen_random_uuid()	PK	
day_id	uuid	yes		FK to production_day, unique	
product_id	uuid	yes		FK to cost_object, unique	
qty_produced	numeric(20,4)				
milk_qty_per_unit	numeric(20,6)				litres of milk embodied in one piece
milk_rate	numeric(20,6)				
conversion_rate	numeric(20,6)				that day's shared cost per litre
material_cost	numeric(20,6)				
conversion_cost	numeric(20,6)				
packing_cost	numeric(20,6)				
other_cost	numeric(20,6)				
unit_cost	numeric(20,6)				material + conversion + packing + other
total_cost	numeric(20,4)				
computed_at	timestampz	yes	now()		
UNIQUE (day_id, product_id)					

Calculation and audit

How the numbers were reached, and who changed what.

calc_run

One row per recalculation: how many values, how long, how many errors. 1 row

Column	Type	Req	Default	Key	Notes
id	uuid	yes	gen_random_uuid()	PK	
scenario_id	uuid	yes		FK to scenario	
status	text	yes	'running'		
node_count	integer				
error_count	integer	yes	0		
duration_ms	integer				
triggered_by	text				
error	text				
started_at	timestamptz	yes	now()		
finished_at	timestamptz				
CHECK ((status = ANY (ARRAY['running'::text, 'ok'::text, 'error'::text])))					

dependency_edge

What reads what. Rebuilt on every calculation, and drives the impact view. 1579 rows

Column	Type	Req	Default	Key	Notes
id	uuid	yes	gen_random_uuid()	PK	
scenario_id	uuid	yes		FK to scenario, unique	
src_ref	text	yes		unique	
dst_ref	text	yes		unique	
UNIQUE (scenario_id, src_ref, dst_ref)					

change_log

Every human edit, old row and new. Recalculations and cache refreshes are deliberately excluded. 4371 rows

Column	Type	Req	Default	Key	Notes
id	bigint	yes	auto	PK	
scenario_id	uuid				
table_name	text	yes			
record_id	uuid				
action	text	yes			
ref	text				
old_row	jsonb				the whole row before the edit
new_row	jsonb				the whole row after the edit
actor	text				
note	text				
changed_at	timestamptz	yes	now()		
CHECK ((action = ANY (ARRAY['insert'::text, 'update'::text, 'delete'::text])))					

Reference and access

Units, and the people who can sign in.

uom

Units of measure, shared across scenarios. 6 rows

Column	Type	Req	Default	Key	Notes
id	uuid	yes	gen_random_uuid()	PK	
code	text	yes		unique	
name	text	yes			
dimension	text	yes			
factor_to_base	numeric(20,10)	yes	1		
base_code	text				
sort_order	integer	yes	0		
UNIQUE (code) · CHECK ((dimension = ANY (ARRAY['mass'::text, 'volume'::text, 'count'::text, 'currency'::text, 'ratio'::text, 'time'::text, 'other'::text])))					

app_user

Sign-in accounts. Passwords are bcrypt hashes; sessions are JWTs in an httpOnly cookie. 1 row

Column	Type	Req	Default	Key	Notes
id	uuid	yes	gen_random_uuid()	PK	
email	text	yes		unique	
name	text	yes			
role	text	yes	'editor'		
created_at	timestampz	yes	now()		
password_hash	text				bcrypt
UNIQUE (email) · CHECK ((role = ANY (ARRAY['owner'::text, 'editor'::text, 'viewer'::text])))					

Views

Read helpers over the tables above. The app reads these rather than reassembling the same joins in a dozen places.

v_cell

One row per object and column, with the effective formula and the last computed value. Mirrors the engine's precedence, so what you read is what it will calculate.

v_daily_summary

One row per day: milk processed, shared costs, the conversion rate they imply, and the production cost that came out.

How a cost is reached

Standard cost. parameter feeds field_def formulas, which compute field_value cells for every cost_object; bom_line pulls one object into another, so a product's cost walks down through its recipe to the milk. dependency_edge records each of those reads, so the app can answer what a change affects.

Daily cost. production_day holds the milk processed; daily_overhead holds what was spent. Their ratio is the day's conversion cost per litre. Each product in daily_production carries that rate in proportion to the milk embodied in it, adds its own material and packing, and the answer is frozen into daily_product_cost so later rate changes leave history alone.